ML Model Deployment Strategies and Model Serving using FastAPI

Dataset: Titanic – Machine Learning from Disaster (Kaggle), train.csv.

Implementation summary

A Random Forest classification pipeline is trained once on a stratified training split. Missing numeric values are imputed with medians, missing categorical values with the most frequent category, and categorical values are one-hot encoded. The fitted preprocessing and classifier are saved together as artifacts/model.pkl and reused for all inference approaches and API serving.

Training and evaluation

Item	Result
Model	Random Forest Classifier
Dataset rows	891
Training rows	712
Testing rows	179
Hold-out accuracy	81.56%

Observations

1. Online vs Batch Inference

Online inference predicts one record at a time and returns an immediate result. Batch inference predicts many records together, which is efficient for larger offline jobs.

2. Streaming vs Batch Inference

Streaming inference handles incoming records sequentially as they arrive, here with a one-second delay. Batch inference collects a group of records and predicts them in one call.

3. Why FastAPI?

FastAPI exposes the model through HTTP REST endpoints, validates request bodies with Pydantic, generates interactive Swagger documentation at /docs, and supports straightforward integration with other applications.

4. Why reuse the same trained model?

Reusing one saved pipeline keeps preprocessing and model parameters consistent across online, batch, streaming, and API predictions. Retraining separate models could produce inconsistent results and makes comparison less reliable.

Run the project with the instructions in README.md. Accuracy and row counts are filled from the actual dataset when model.py is run.